Payment Abstraction V2

Table of contents

- Payment Abstraction V2
  - Table of contents
  - Goals
  - Anti-Goals
  - Glossary
- Design
  - Dutch Auction Model
    - Curve
    - Auction kick-off and termination
  - CowSwap Integration
  - Streams Integration
  - Auction Bidder
  - Workflow Router
  - System Parameters
    - Feed Info
    - Asset Parameters
    - State
    - Roles
  - Sequence Diagram

Goals

The goal of Payment Abstraction V2 is to replace the current Payment Abstraction V1 system by:

- Implementing a permissionless dutch auction mechanism where any participant can bid on auctioned allowlisted assets. This model improves flexibility and resiliency by supporting multiple liquidity venues.
- Integrating with CowSwap to relay auctions to the protocol's solvers to ensure sufficient participation.

- Building a backward and forward compatible solution allowing to integrate with multiple liquidity venues.

Glossary

- **Solvers**: auction participants e.g. CowSwap solvers or any market participants bidding on the auctions.
- **Price multiplier**: the auction price multiplier at a given elapsed time.
- **Dutch auction**: type of auction in which an auctioneer starts with a very high price, incrementally lowering the price until someone places a bid.
- **CRE Workflow**: executable binaries deployed on a Decentralized Oracle Network (DON) via the Chainlink Runtime Environment (CRE).
- **Payment Abstraction V1**: first iteration of Payment Abstraction which leverages Uniswap V3 for swaps.

Design

Dutch Auction Model

Curve

For the auction price decay curve is linear such as:

$$P = P_{max} - \frac{D_{max}}{t}$$

P: $price\ multiplier\ e.g.\ 0.98$

D_{max}: $maximum\ discount\ e.g.\ 0.02\ (max\ slippage)$

t: $elapsed\ time$

P_{max}: $max\ price\ multiplier\ e.g.\ 1\ (auction\ start\ value)$

On-chain example (see asset parameters):

- We auction 100,000 USDC
- 1 USDC = $1
- 1 LINK = $20
- Auction LINK value = 100,000/20 = 5,000 LINK
- Auction duration: 2 hours
- Starting price multiplier: 1.05e18 (5% premium)

- Maximum discount of 2% (i.e 0.98e18)
- Decay rate per second = (1.05e18 - 0.98e18) / (60 * 60 * 2) = 9722222222222 (we round down in favor of the system here to avoid a higher discount than the maximum)
- Auction start value = 5000 * (1.05 - 0.000009722222222222 * 0) = 5250 LINK
- Auction value at 1 hour = 5000 * (1.05 - 0.000009722222222222 * 60 * 60) = 5075.000000000004 LINK
- Auction end value = 5000 * (1.05 - 0.000009722222222222 * 60 * 60 * 2) = 4900.000000000008 LINK

This computation is performed by the getAssetOutAmount function of the auction contract. All rounding operations are done in the protocol's favor, which given the contract's preconditions, guarantees a non zero value:

Variable	Precondition	Enforced by
amountIn	> 0	Min bid USD value check in bid() and amount check in isValidSignature()
assetInUsdPrice	> 0	_getAssetPrice() validation
assetInDecimals	> 0	Asset param config validation
pricemultiplier	> 0	Bounded between endingPriceMultiplier and startingPriceMultipler, both > 0
assetOutDecimals	> 0	Asset param config validation
assetOutUsdPrice	> 0	_getAssetPrice() validation

- Step 1: mulDivUp(amountIn, assetInUsdPrice, 10^assetInDecimals)
  amountIn > 0 && assetInUsdPrice > 0 → amountIn * assetInUsdPrice > 0 → ⌈x / y⌉ ≥ 1
- Step 2 — mulWadUp(step1, priceMultiplier)
  step1 ≥ 1 && priceMultiplier > 0 → step1 * priceMultiplier > 0 → ⌈x / 1e18⌉ ≥ 1
- Step 3 — mulDivUp(step2, 10^assetOutDecimals, assetOutUsdPrice)
  step2 ≥ 1 && 10^assetOutDecimals > 0 → step2 * 10^assetOutDecimals > 0 → ⌈x / y⌉ ≥ 1

So assetOutAmount ≥ 1 for all valid inputs.

Auction kick-off and termination

To start and end auctions, we use a CRE workflow which, through the checkUpkeep/performUpkeep functions, monitors the FeeAggregator for available allowlisted assets balances and the auction contract for ended auctions.

For an auction to be considered eligible to start, the following conditions must be met:

- The asset out price must be valid (not stale and not zero).
- There must not already be a live auction for the asset.
- The asset price is valid (not stale and not zero).
- The total USD value of the asset (in both the fee aggregator and the auction contract) is greater or equal to the minimum auction size.

For an auction to be considered ended, either of the following conditions must be met:

- The auction duration has elapsed since the auction start time.
- The total USD value of the asset remaining in the auction contract is below the minimum auction size. This is to guard against dust attacks.

Although the system is compatible with Chainlink Automation (CLA), the checkUpkeep and performUpkeep functions will be called through a CRE workflow not CLA. CLA can be used as a potential fallback.

The performUpkeep function is access controlled to only addresses that are granted the AUCTION_WORKER_ROLE which is a trusted role. Therefore not all checkUpkeep checks are re-performed in performUpkeep:

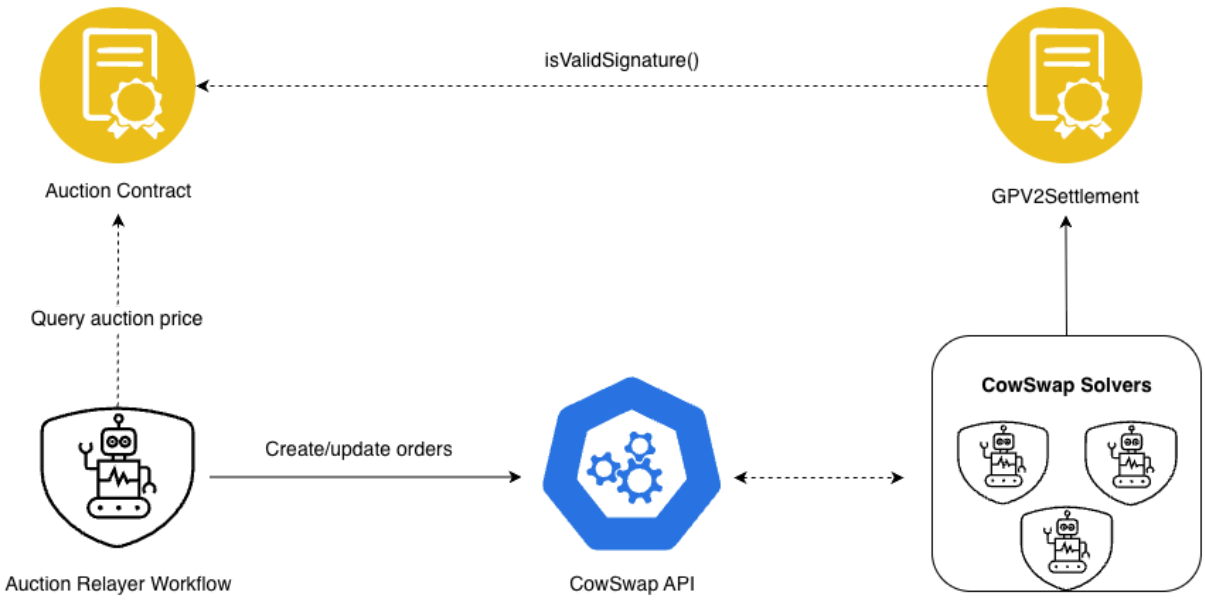
Check	checkUpkeep	performUpkeep	Description
Paused	Yes	Yes	Contract must not be paused
Configured asset out	Yes	Yes	Asset out must be configured
Access control (AUCTION_WORKER_ROLE)	Yes	Yes	Caller must have the AUCTION_WORKER_ROLE
Configured asset	Soft (skip)	Hard (revert)	The auctioned asset must be configured
Valid asset out price	Soft (zeros eligible list)	Hard (revert)	The asset out price must not be stale or zero

Valid asset price	Soft (skip)	Hard (revert)	The auctioned asset price must not be stale or zero
Above min auction size (start)	Soft (skip)	Hard (revert)	The USD value of the auctioned asset must meet the minimum threshold
No existing live auction (start)	Soft (skip)	Hard (revert)	Only assets without a live auction are eligible to start
Elapsed auction duration (end)	Yes	No (allows for forced endings)	Only auctions that have elapsed their full duration are eligible to end
Below min auction size (end)	Yes	No (allows for forced endings)	Only auctions that have a remaining asset balance's USD value below the minimum threshold are eligible to end
Existing auction (end)	Implicit	Hard (revert)	Only live auctions are eligible to end

CowSwap Integration

CoW Protocol uses an off-chain order book to collect signed messages before they become orders, allowing for additional optimizations. This canonically requires pre-signing the order from a wallet off-chain to allow the on-chain execution. However, Cow Protocol also supports ERC-1271 for smart contracts to be able to trade using the system. The order verification logic is handled with EIP-1271's isValidSignature and the actual enforcement is handled at the CowSwap level.

The system therefore implements a CRE workflow that monitors the live auction prices on the Auction contract and posts partially fillable limit orders on the CowSwap API.



Streams Integration

Payment Abstraction V2 integrates with Chainlink Data Streams to be able to push prices to the Auction contract directly and at a higher frequency during auctions than Data Feeds. The contract still falls back on Data Feeds if there are no fresh Streams prices. A CRE workflow is used for this purpose which:

- Queries the Streams API for prices
- Pushes them to the Auction contract

Auction Bidder

For future deployments on chains where CowSwap may not be available and/or to operate as a fallback, other solutions (e.g. DEX aggregators) can be used through the AuctionBidder, which includes:

- A smart contract to handle auction callbacks with arbitrary execution (AuctionBidder)
- A CRE workflow to push arbitrary logic execution to the AuctionBidder contract

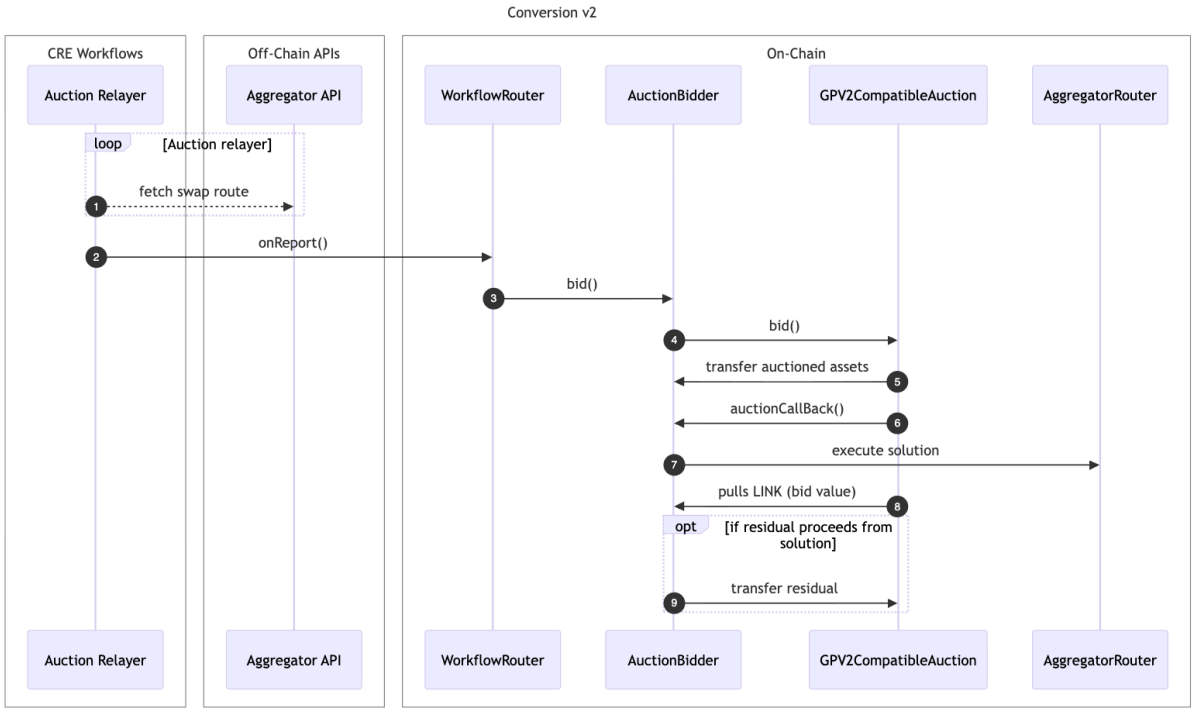
Example:

- CRE workflow queries an aggregator's swap route and pushes it to the solver contract

- The AuctionBidder contract bids on the auction and executes swap on the callback

External market participants can also deploy their own instance of the AuctionBidder to participate in auctions. An optional receiver can be set to receive proceeds from positive slippage when solutions yield a better rate than the auction value and any funds can be accessed by the contract admin through the withdraw function. In our case this receiver will be set to the auction contract.

Call flow:



1. The auction bidder workflow fetches the swap route from the aggregator off-chain API
2. The auction bidder workflow pushes the swap route through the chain write capability which calls onReport on the WorkflowRouter
3. The WorkflowRouter routes the call to the AuctionBidder's bid function
4. The AuctionBidder calls the auction contract's own bid functions
5. The auction contract transfers the auctioned assets to the AuctionBidder
6. The auction contract calls the AuctionBidder's callback function with the relayed swap route
7. The AuctionBidder executes the swap route by calling the aggregator router and approves the proceeds to be spent by the auction contract
8. The auction contract pulls the amount of LINK required by the auction curve

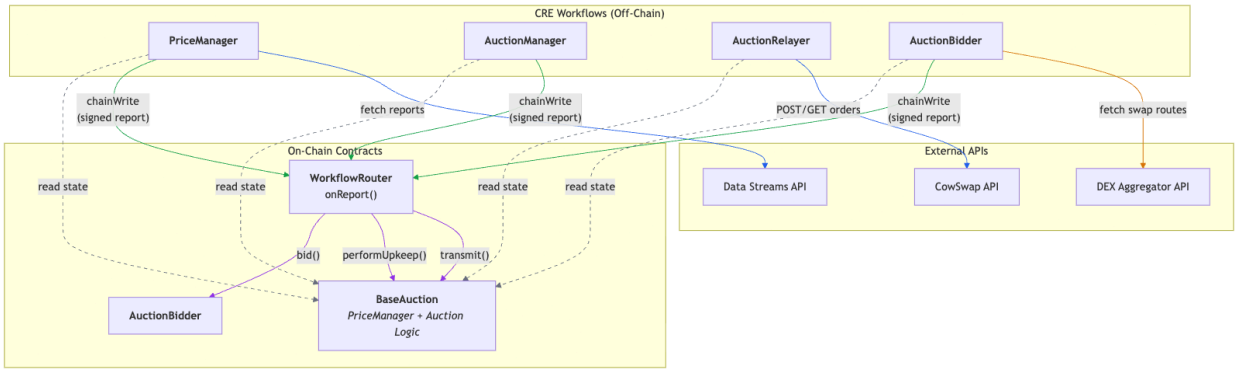
9. If there are any residuals (positive slippage), the auction bidders transfers them to the auction contract

Workflow Router

Integrating with CRE has some on-chain implications ([Building Consumer Contracts | Chainlink Documentation]). Since there are multiple workflows interacting with multiple contracts/functions namely:

- Auction contract:
  - checkUpkeep / performUpkeep for auction management
  - transmit for price transmission
- Solver contract:
  - bid for auction participation

And for increased modularity, on-chain workflow validation and routing is performed by an intermediate WorkflowRouter contract.



System Parameters

Some configuration validations are intentionally performed off-chain through deployment and operational tooling rather than enforced on-chain. This approach avoids unnecessary gas overhead for checks that are only relevant at configuration time and are performed by trusted, role-gated callers. These off-chain validations include, but are not limited to:

- Enforcing sensible bounds on staleness thresholds, auction durations, price multiplier spreads, and minimum bid USD values
- Verifying that data feed addresses and Data Streams feed IDs correspond to the intended assets
- Ensuring allowlists remain synchronized between the auction contract and FeeAggregator

- Confirming that newly configured assets have valid price feeds
- Validating that receiver addresses and fee aggregator registrations are correctly set up prior to use.

Feed Info

Parameter	Description
dataStreamsFeedId	Unique identifier for the data stream.
usdDataFeed	Address of the USD data feed.
stalenessThreshold	Maximum age of the price data (seconds).
dataStreamsFeedDecimals	Number of decimals in the reported data streams price.

Asset Parameters

Parameter	Description
minAuctionSizeUsd	The minimum auction size expressed in USD feed decimals. Determines if the auction management workflow can pull funds from the FeeAggregator and start the auction.
startingPriceMultiplier	The starting price multiplier with 18 decimals precision.
endingPriceMultiplier	The ending price multiplier with 18 decimals precision. Example: • startingPriceMultiplier = 1.1e18 (10% premium) • endingPriceMultiplier = 0.98e18 (2% discount) • auctionDuration = 3600 (1 hour) • decayRatePerSecond = (1.1e18 - 0.98e18) / 3600 = 33333333333333 (rounded down to avoid lower discount than 2%)
auctionDuration	The duration of the auction in seconds.
decimals	The asset decimals. We store this in state to save an external call on the hot path.

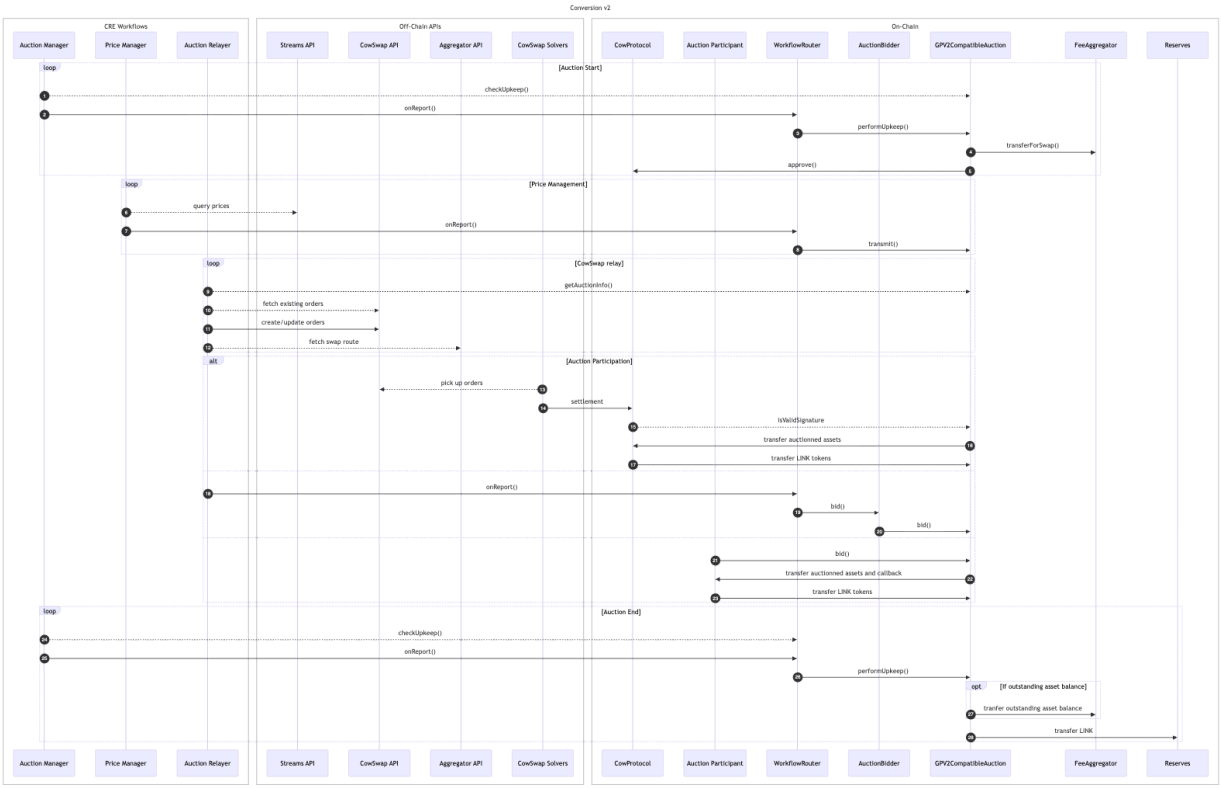
State

Variable	Description
s_allowlistedAssets	List of auctionable assets.
s_assetOut	The asset out of all the auctions (LINK for PAL usecase).
s_assetOutReceiver	The receiver of assets out (Reserves.sol for PAL usecase).
s_auctionStarts	The live auctions start timestamp

Roles

Role	Description	Granted To
DEFAULT_ADMIN_ROLE	Owner of the contracts. Manages roles, critical configurations and can emergency withdraw assets.	Timelock
PAUSER_ROLE	Pauses the contract when an emergency is detected.	Monitoring
UNPAUSER_ROLE	Unpauses the contract when an emergency is resolved. This is separate from the PAUSER_ROLE to decouple the role.	Timelock
SWAPPER_ROLE	Assigned to the GPV2CompatibleAuction contract, this role allows for pulling allowlisted assets from the FeeAggregator contract when starting an auction.	GPV2CompatibleAuction
ASSET_ADMIN_ROLE	Configures feeds, asset parameters, minimum bid value and asset out.	Timelock
PRICE_ADMIN_ROLE	Transmits Streams prices to the GPV2CompatibleAuction contract.	WorkflowRouter
AUCTION_WORKER_ROLE	Starts and expires auctions by calling performUpkeep on the GPV2CompatibleAuction contract.	WorkflowRouter
AUCTION_BIDDER_ROLE	Pushes solutions to the AuctionBidder's bid function to solve live auctions.	WorkflowRouter
ORDER_MANAGER_ROLE	Invalidates orders on the GPv2Settlement contract.	WorkflowRouter
FORWARDER_ROLE	Transmits CRE workflow payloads to the WorkflowRouter's onReport function.	CRE forwarder

Sequence Diagram



1. The auction manager workflow calls the checkUpkeep function on the auction contract. The checkUpkeep function:
    a. Determines the USD values of auctionable assets balances
    b. Returns the list of auctionable assets that have sufficient balances to meet the auction minimum USD value
2. If checkUpkeep returns true, the auction manager workflow calls onReport on the WorkflowRouter with the auction manager workflow id and the perform data through the chain write capability.
3. The WorkflowRouter routes the workflow data to the auction contract's performUpkeep function which starts auctions by setting the initial price and the timestamp
4. Calls transferForSwap on the FeeAggregator to pull the assets to auction
5. Approve amounts to the CowSwap Vault relayer
6. The price manager workflow calls getAuctionStart for each allowlisted asset on the auction contract to detect live auctions. If a live auction is detected and the last price update is older than the threshold, it fetches new prices from the Streams API.

7. The price manager workflow pushes fresh prices and adjusts its price update frequency. Prices are pushed through the CRE chain-write capability which calls onReport on the WorkflowRouter.
8. The WorkflowRouter routes the workflow data to the auction contract's transmit function.
9. The CowSwap relayer workflow also monitors live auctions with the same logic as the price manager workflow.
10. When it detects live auctions, it fetches the latest orders posted on the CowSwap API for the auctioned assets.
11. This is to determine whether an order should be created or updated.
    a. If the order exists and its creation timestamp is older than the auction start, then a new order is created for the full auction amount, at the current auction price.
    b. If the order exists and its creation timestamp is greater than the auction start, then, the order is updated if:
        i. If the current auction price is lower than the order price by at least x% (TBD)
        ii. If the available auction amount is lower than the order amount

    Note: orders updates can be handled either by:

    - Invalidating the order on-chain through the WorkflowRouter and creating a new order
    - By creating short lived orders, therefore not requiring an on-chain transactions as the order will expire automatically
12. Optionally, if the internal solver is enabled, the workflow fetches the swap route from the configured aggregator
13. The CowSwap solvers pick up the order
14. And submit it to the CowSwap settlement contract
15. The CowSwap contract calls EIP-1271 isValidSignature on the Auction contract with the order details which the Auction contract validates. If the order parameters are valid then it returns the valid 1271 value.
16. The CowSwap contract can then pull the assets from the auction contract
17. And transfer the LINK tokens
18. If the internal solver is enabled, the auction relayer workflow pushes the aggregator swap route to the WorkflowRouter through the onReport function
19. The WorkflowRouter routes the swap route to AuctionBidder's bid function.
20. The AuctionBidder solves the auction by calling the auction's contract bid function.
21. Regular market participants will also be competing against CowSwap solvers and the internal solver. If any of them finds an opportunity first, they can call the bid function on the Auction contract.

22. Which transfer the tokens to the bidder and optionally execute a callback
23. And pull the LINK tokens
24. To end auctions the auction management workflow calls checkUpkeep again which will add asset addresses to the ended auction list in the perform data if:
    a. The auction has expired (i.e. if auction start timestamp + auction duration > block.timestamp)
    b. Or if the auctioned asset balance is below the minimum auction size
25. The performed data is relayed through the WorkflowRouter onReport function
26. The WorkflowRouter routes the call the auction contract's performUpkeep function
27. If there is an outstanding balance of auctioned assets, it is transferred back to the FeeAggregator
28. The auction proceeds (LINK) are sent to the set receiver (Reserves)